Ministry of Education, Culture and Research of the Republic of Moldova
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automatic

ANDREI ELCISCEV
Group FAF-243

LABORATORY WORK No. 9
WATCHING SYSTEM CALLS WITH STRACE

Checked by:
Reiţman Patricia

Chişinău, 2026

Contents

1 Introduction **2**

2 Experimental Environment **2**

3 Part 1: Watching a Real Command **2**
3.1 Tracing `ls -l` 2
3.2 Observe: Printing to the Screen 3
3.3 Rough Number of Calls 4

4 Part 2: A Program Using File Descriptors **5**
4.1 Source Code 5
4.2 Compilation and Execution 5
4.3 Inspecting the Actual Calls 6
4.4 Observe: Why Descriptor 3? 7
4.5 Checking the Result 7

5 Part 3: Counting One Kind of Call **7**

6 Results and Discussion **8**

7 Conclusion **8**

1 Introduction

This laboratory work investigates the interface between Linux applications and the kernel using `strace`. The experiments trace a familiar command, compile a small C program that writes and reads a file, and count selected system calls. The traces connect file descriptors, byte counts, and return values with the operations performed by each program.

All experiments were executed in the Linux Mint 22.3 VirtualBox VM, under the `user` account, in `/home/user/os-lab9`. The directory was created before the lab exercises. Observe answers are stored in `report.md` there. The listings below contain actual captured commands and output in place of screenshots. Long lines may wrap for readability. Timing values describe individual measurements and are not performance benchmarks.

The supplied two-page assignment lists Part 3, "Count one kind of call," in its overview, but contains detailed instructions and Observe questions only for Parts 1 and 2. Section 5 therefore records a supplementary experiment covering that stated goal; no missing questions are assumed.

2 Experimental Environment

```
$ pwd; uname -sr; gcc --version | head -1; strace --version | head -1
/home/user/os-lab9
Linux 6.14.0-37-generic
gcc (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0
strace -- version 6.8
```

Listing 1: Working directory, kernel, compiler, and tracing tool

The required compiler and tracing utility were already installed. The shell environment was set to `LC_ALL=C` for consistent English output. Commands were executed through VirtualBox guest control, with standard output captured to files rather than attached to an interactive terminal. This matters for standard I/O buffering, discussed below.

3 Part 1: Watching a Real Command

3.1 Tracing `ls -l`

The option `-f` follows child processes if any are created. The expression `-e trace=openat,read,write,close` limits the displayed calls to those four names. Consequently, this trace does not show every call made by `ls`. In particular, directory enumeration uses `getdents64`, which this filter does not display. Opening libraries, reading configuration and user/group data, and printing the listing also require kernel services.

```
$ strace -f -e trace=openat,read,write,close ls -l
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
close(3)                                = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libselinux.so.1", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0"..., 832) =
    832
close(3)                                = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
```

```
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0\0"..., 832)
    = 832
close(3)                                = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libpcre2-8.so.0", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0"..., 832) =
    832
close(3)                                = 0
openat(AT_FDCWD, "/proc/filesystems", O_RDONLY|O_CLOEXEC) = 3
read(3, "nodev\tsysfs\nnodev\ttmpfs\nnodev\tbd"..., 1024) = 400
read(3, "", 1024)                       = 0
close(3)                                = 0
openat(AT_FDCWD, ".", O_RDONLY|O_NONBLOCK|O_CLOEXEC|O_DIRECTORY) = 3
close(4)                                = 0
close(4)                                = 0
openat(AT_FDCWD, "/etc/nsswitch.conf", O_RDONLY|O_CLOEXEC) = 4
read(4, "# /etc/nsswitch.conf\n#\n# Example"..., 4096) = 569
read(4, "", 4096)                       = 0
close(4)                                = 0
openat(AT_FDCWD, "/etc/passwd", O_RDONLY|O_CLOEXEC) = 4
read(4, "root:x:0:0:root:/root:/bin/bash\n"..., 4096) = 2876
close(4)                                = 0
close(4)                                = 0
close(4)                                = 0
openat(AT_FDCWD, "/etc/group", O_RDONLY|O_CLOEXEC) = 4
read(4, "root:x:0:\ndaemon:x:1:\nbin:x:2:\ns"..., 4096) = 1148
close(4)                                = 0
close(3)                                = 0
openat(AT_FDCWD, "/etc/localtime", O_RDONLY|O_CLOEXEC) = 3
read(3, "TZif2\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\20\0\0\0\0"..., 4096) =
    2390
read(3, "TZif2\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\20\0\0\0\0"..., 4096) =
    1491
close(3)                                = 0
write(1, "total 8\ndrwxrwxr-x 2 user user 4"..., 113total 8
drwxrwxr-x 2 user user 4096 Nov  2 16:17 evidence
-rw------- 1 user user 1742 Nov  2 16:17 experiment.sh
) = 113
close(1)                                = 0
close(2)                                = 0
+++ exited with 0 +++
```

Listing 2: Actual filtered trace of ls, including its normal output

The apparent mixing of the directory listing into a `write` line occurs because normal output and `strace`'s diagnostic stream were captured together. It is a recording effect, not an invalid system call.

3.2 Observe: Printing to the Screen

Question: Which system call does `ls` use to print to the screen? To which descriptor does it write?

Answer: It uses `write`, writing to descriptor `1`, standard output. In the captured run, the trace shows a 113-byte write to descriptor 1. When run interactively, standard output normally points to the terminal; in this experiment it was redirected into the evidence file.

3.3 Rough Number of Calls

```
$ strace -f -c -e trace=openat,read,write,close ls -l
total 32
drwxrwxr-x 2 user user  4096 Nov  2 16:17 evidence
-rw------- 1 user user  1742 Nov  2 16:17 experiment.sh
-rw-r--r-- 1 user user    14 Nov  2 16:17 note.txt
-rwxrwxr-x 1 user user 16272 Nov  2 16:17 syscalls
-rw-rw-r-- 1 user user   663 Nov  2 16:17 syscalls.c
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 52.71    0.001564          97        16           close
 23.19    0.000688          62        11           read
 21.30    0.000632          63        10           openat
  2.80    0.000083          83         1           write
------ ----------- ----------- --------- --------- ----------------
100.00    0.002967          78        38           total
```

Listing 3: Summary restricted to the four selected system calls

This measurement reports 38 selected calls: 10 `openat`, 11 `read`, 16 `close`, and one `write`. This is a filtered count, not the total number of system calls. A second measurement without a syscall filter gives the full summary:

```
$ strace -f -c ls -l
total 32
drwxrwxr-x 2 user user  4096 Nov  2 16:17 evidence
-rw------- 1 user user  1742 Nov  2 16:17 experiment.sh
-rw-r--r-- 1 user user    14 Nov  2 16:17 note.txt
-rwxrwxr-x 1 user user 16272 Nov  2 16:17 syscalls
-rw-rw-r-- 1 user user   663 Nov  2 16:17 syscalls.c
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 15.00    0.002673         243        11           read
 14.68    0.002617         163        16           close
 14.47    0.002580         151        17           mmap
 10.27    0.001831         183        10           openat
  7.35    0.001310         262         5           statx
  5.69    0.001015         338         3           newfstatat
  5.22    0.000931          71        13           fstat
  3.78    0.000674         674         1           execve
  3.32    0.000592         592         1           getrandom
  3.10    0.000553         110         5         5 lgetxattr
  3.05    0.000544         136         4         4 connect
  2.05    0.000365          73         5           mprotect
  2.01    0.000359         119         3           brk
  1.74    0.000311         311         1           arch_prctl
  1.51    0.000269          53         5           listxattr
  1.46    0.000260          65         4           socket
  1.35    0.000240         120         2         2 statfs
  0.74    0.000132         132         1           write
  0.72    0.000129         129         1           munmap
  0.51    0.000091          30         3           lseek
  0.47    0.000084          42         2         2 access
  0.39    0.000070          35         2           pread64
  0.30    0.000054          54         1           rseq
  0.30    0.000053          53         1           set_tid_address
```

```
  0.29    0.000051          51         1         1 ioctl
  0.12    0.000022          11         2           getdents64
  0.06    0.000011          11         1           prlimit64
  0.02    0.000003           3         1           set_robust_list
------ ----------- ----------- --------- --------- ----------------
100.00    0.017824         146       122        14 total
```

Listing 4: Unfiltered summary of system calls made by ls

The unfiltered measurement reports 122 calls, including two `getdents64` calls. Thus, a simple long listing makes roughly a hundred calls in this setup. The directory had more files by the time of these summaries than during the first trace because the program had been created and compiled. Counts depend on directory contents, environment, libraries, and output destination. The 14 errors in the summary are individual failed operations, including optional resource probes; they do not mean that `ls` itself failed.

4 Part 2: A Program Using File Descriptors

4.1 Source Code

The supplied program was saved as `syscalls.c`. It opens `note.txt`, writes a message, closes the descriptor, opens the file again, and reads the message back. File contents are transferred with `write` and `read`; `printf` reports the results on standard output.

```
$ cat syscalls.c
/* syscalls.c - open, write, read and close. Lab 9 supplied program. */
#include <fcntl.h>
#include <unistd.h>
#include <string.h>
#include <stdio.h>

int main(void) {
    const char *msg = "hello, kernel\n";
    int fd = open("note.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (fd < 0) { perror("open"); return 1; }
    printf("open returned descriptor %d\n", fd);
    ssize_t n = write(fd, msg, strlen(msg));
    printf("write sent %zd bytes\n", n);
    close(fd);
    fd = open("note.txt", O_RDONLY);
    char buf[64];
    n = read(fd, buf, sizeof(buf) - 1);
    buf[n] = '\0';
    printf("read got %zd bytes: %s", n, buf);
    close(fd);
    return 0;
}
```

Listing 5: Source code compiled in the VM

4.2 Compilation and Execution

```
$ gcc -Wall -Wextra -o syscalls syscalls.c; ./syscalls; wc -c note.txt; cat note.txt
```

```
open returned descriptor 3
write sent 14 bytes
read got 14 bytes: hello, kernel
14 note.txt
hello, kernel
```

Listing 6: Compilation, execution, and inspection of the written file

Compilation with `-Wall -Wextra` produced no diagnostics. The message contains 13 visible characters plus a newline, so its length is 14 bytes. `strlen` excludes the terminating null byte. The read requests up to 63 bytes and returns 14, the number actually available in the file.

4.3 Inspecting the Actual Calls

The required filtered trace was executed directly. For a more readable recording, it was repeated with `-o`, which places the trace in a separate file while leaving the program's standard output separate. Both outputs are reproduced below. The unfiltered command `strace ./syscalls` was also run and saved in `evidence/10_full_trace.txt` in the VM.

```
$ strace -o evidence/syscalls-clean.trace -e trace=openat,write,read,close ./syscalls
open returned descriptor 3
write sent 14 bytes
read got 14 bytes: hello, kernel
$ cat evidence/syscalls-clean.trace
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
close(3)                                = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0\0"..., 832)
    = 832
close(3)                                = 0
openat(AT_FDCWD, "note.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 3
write(3, "hello, kernel\n", 14)         = 14
close(3)                                = 0
openat(AT_FDCWD, "note.txt", O_RDONLY)  = 3
read(3, "hello, kernel\n", 63)          = 14
close(3)                                = 0
write(1, "open returned descriptor 3\nwrite"..., 80) = 80
+++ exited with 0 +++
```

Listing 7: Program output followed by the separately captured filtered trace

The initial opens and read belong to dynamic-loader setup. The application then opens `note.txt` with creation, write-only, and truncation flags and obtains descriptor 3. Its `write(3,...)` returns 14. Closing the file frees the descriptor, so opening it again can return 3. The subsequent `read` returns the same 14 bytes, followed by another successful close.

Although the C source calls `open`, the C library implements that operation through the Linux `openat` syscall here. `AT_FDCWD` resolves a relative pathname against the process's current working directory.

4.4 Observe: Why Descriptor 3?

Question: The descriptor returned by the first `openat` is 3, not 0, 1, or 2. Why are those three already taken?

Answer: The process inherits three standard descriptors: 0 is standard input, 1 is standard output, and 2 is standard error. At the point where `note.txt` is opened, these are occupied and 3 is the lowest available descriptor. The temporary loader descriptor 3 has already been closed.

4.5 Checking the Result

```
$ test "$(wc -c < note.txt)" -eq 14 && printf "PASS: note.txt contains 14 bytes\n";
    printf "hello, kernel\n" | cmp - note.txt && printf "PASS: file content matches
    the written message\n"
PASS: note.txt contains 14 bytes
PASS: file content matches the written message
```

Listing 8: Checks of the actual byte count and exact file contents

The original teaching program was retained. It checks the first open but does not check the second open, the read result, or partial writes. In production code, these must be checked before using `buf[n]` or assuming the whole message was written. Both operations succeeded in the recorded experiment.

5 Part 3: Counting One Kind of Call

This supplementary experiment uses `-c` to summarize calls and `-e trace=write` to count only writes.

```
$ strace -c -e trace=write ./syscalls
open returned descriptor 3
write sent 14 bytes
read got 14 bytes: hello, kernel
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
100.00    0.000061          30         2           write
------ ----------- ----------- --------- --------- ----------------
100.00    0.000061          30         2           total
```

Listing 9: Counting write calls made by the program

There are two `write` calls: one sends 14 bytes to `note.txt` through descriptor 3, and one sends 80 bytes of status messages to standard output through descriptor 1. The three `printf` calls do not necessarily produce three kernel calls. Because output is captured through a non-terminal stream, the C library buffers those messages and flushes them together at program exit. Interactive terminal execution can produce a different count because standard output is normally line buffered there.

```
$ strace -c -e trace=openat,write,read,close ./syscalls
open returned descriptor 3
write sent 14 bytes
read got 14 bytes: hello, kernel
% time     seconds  usecs/call     calls    errors syscall
```

```
------ ----------- ----------- --------- --------- ----------------
 60.28    0.000900         225         4           openat
 18.82    0.000281         140         2           write
 16.14    0.000241          60         4           close
  4.76    0.000071          35         2           read
------ ----------- ----------- --------- --------- ----------------
100.00    0.001493         124        12           total
```

Listing 10: Summary of the four selected calls for the program

The selected total is 12 calls: four `openat`, two `read`, two `write`, and four `close`. These include loader activity as well as the program's own file operations. The six calls specifically operating on `note.txt` are two opens, one write, one read, and two closes.

6 Results and Discussion

The traces demonstrate that visible program actions can involve more kernel calls than the source suggests. A directory listing loads libraries, obtains metadata, looks up user and group names, and enumerates directory entries before printing. Filtering helps isolate relevant calls but must not be mistaken for a complete count.

The C program confirmed the relationship between descriptors and open files. The same descriptor number was reused after closing the file; successful writes and reads returned the number of transferred bytes. The correspondence between `open` and `openat`, and between multiple `printf` calls and one buffered `write`, also showed why library calls and system calls are not always in one-to-one correspondence.

7 Conclusion

This laboratory work provided practical experience tracing Linux system calls, interpreting descriptors and return values, and counting selected operations. The program successfully wrote and read back the 14-byte message. The observations explain standard descriptors, descriptor reuse, library startup activity, and the effect of output buffering on measured calls. Actual command transcripts preserve the experimental evidence without external screenshots or other report assets.